

ONLINE SESSION · 2.5 HOURS

Session 19

Security — Part 1

Keycloak & OAuth2

Identity Provider · Authorization Code Flow · Realms, Clients, Users



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com



Wednesday · 3:00–5:30 PM

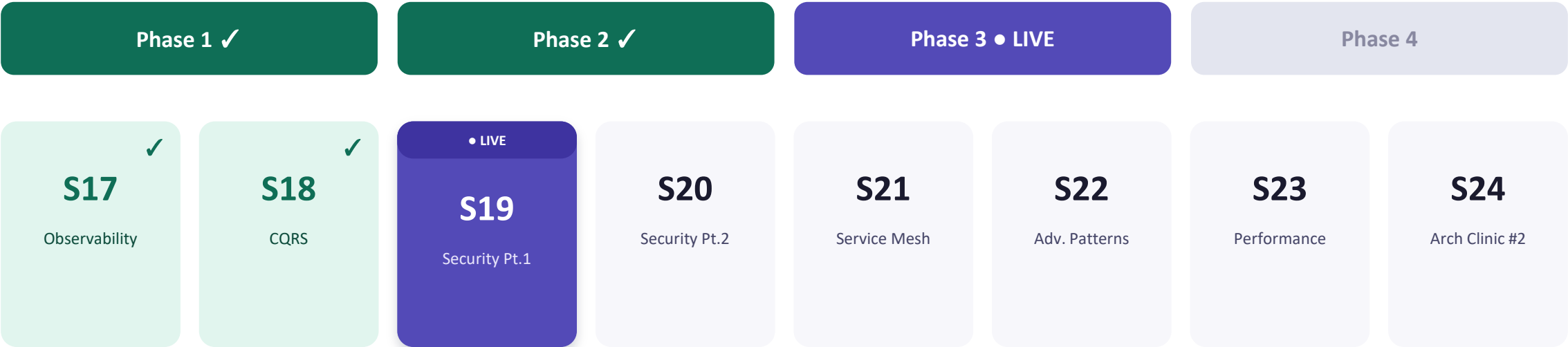


Phase 3 — Advanced & Enterprise



Session 19 of 29

Phase 3 — Advanced & Enterprise



Session 19 answers the question Session 3 deliberately left open: where do real, trustworthy tokens actually come from?

Note: the full 29-session roadmap lives in the Kickoff deck only.

Answering the Question Session 3 Left Open

"In Session 3 we built JwtAuthFilter — it validated a JWT's signature using a shared secret hardcoded in application.yml. We even wrote a production note at the time: *'In a real system, Keycloak or Auth0 issues the tokens. The Gateway only validates — it does not issue tokens. We cover the full OAuth2 flow in a later session.'*"

Today is that session — for the human-login half of the story.

Where do real tokens come from, who is a real user, and how does a browser-based login actually work end to end?

🔗 **Explicit scope:** Today covers human login only. Session 20 covers service-to-service authentication (Client Credentials) and wiring the Gateway as a real OAuth2 Resource Server.

3 Questions Session 3's Security Cannot Answer

1

Where do real customers create accounts and set passwords?

There is no signup form — we only ever generated test tokens with a Java snippet.

2

A customer forgets their password. What happens?

Nothing — we built no reset flow, because there is no real user store.

3

A customer's laptop is stolen with an active session. Can you revoke JUST their token?

No — JwtAuthFilter only checks a signature and an expiry timestamp; it has no concept of 'this specific token is now invalid.'

From Shared Secret to Trusted Issuer



Session 3

Trust because I know the secret

The Gateway validates a JWT signature using a secret hardcoded in application.yml. Anyone with that secret can forge a valid token.



Session 19

Trust because I trust the issuer

The Gateway trusts tokens signed by Keycloak — a real Identity Provider. Complexity of authentication lives in ONE place.

This is the same 'trust the boundary' principle from Session 3's Gateway-as-trust-boundary lesson — just one layer further out.

Session 3: services trust the Gateway. Today: the Gateway trusts Keycloak.

The Four OAuth2 Roles

Role	Who It Is in Our Platform	Responsibility
Resource Owner	The customer using the E-Commerce app	Owns the data/account; grants or denies access
Client	The API Gateway	Requests access on the Resource Owner's behalf
Authorization Server	Keycloak	Authenticates the Resource Owner; issues tokens
Resource Server	Order/Product Service (via Gateway)	Accepts tokens; serves protected resources

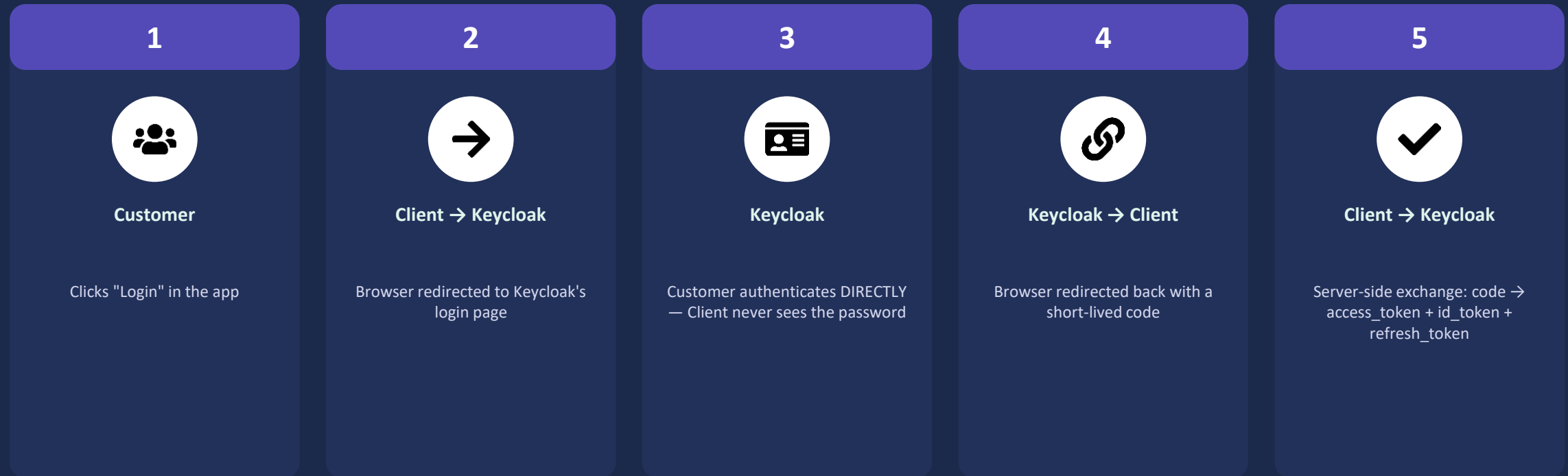
Identity Provider — Trust the Boundary, One Layer Out

An Identity Provider (IdP) is a system whose entire job is proving "this really is who they claim to be" and issuing a signed, trustworthy statement of that fact (the token).

Everyone else — our Gateway, our services — simply trusts tokens signed by the IdP, without needing to know HOW the user actually authenticated (password? social login? multi-factor?). That complexity lives in ONE place: Keycloak.

🔗 Ask: "If you were the Gateway, would you rather trust a token because you know the secret that signed it, or because you trust WHO signed it?"

Authorization Code Flow — Step by Step



Key message: notice steps 2–4 happen entirely through browser redirects — the Client application code is not involved until step 5.

Why Redirects — Not a Direct API Call?

This is deliberate: the customer's password is typed only into Keycloak's own page, never into our Gateway or any of our code.

If the Client collected the password and sent it via API call, the Client would have full access to raw credentials — a much larger attack surface, and it defeats the purpose of centralizing identity in one trusted place.

The core security property of the Authorization Code Flow:

The Client only ever receives a short-lived authorization code — never the password, never the raw credentials.

Authorization Code vs Client Credentials

User logs in, human interaction?	→	Authorization Code (today)
Service-to-service, no user?	→	Client Credentials (Session 20)
Mobile/SPA app?	→	Authorization Code + PKCE
Backend API called by another service?	→	Client Credentials (Session 20)

Today we build ONLY the human-login half. Client Credentials is Session 20's explicit scope — not covered here.

Public Client vs Confidential Client

Server-side app that can safely store a secret? (our Gateway)



Confidential Client — has a `client_secret`

SPA or mobile app — code visible to the end user?



Public Client — no secret; use PKCE instead

Need the strongest guarantee only the real Client exchanges a code?



Confidential Client + `client_secret`

Browser-only JavaScript frontend, no backend?



Public Client + PKCE (beyond today's depth)

Our platform: the API Gateway is a Confidential Client — it runs entirely server-side and can safely hold a `client_secret`.

Add Keycloak to docker-compose.yml

```
# docker-compose.yml - new service
keycloak:
  image: quay.io/keycloak/keycloak:24.0
  command: start-dev
  environment:
    - KEYCLOAK_ADMIN=admin
    - KEYCLOAK_ADMIN_PASSWORD=admin # DEV ONLY - never a real password in prod
  ports: ["8090:8080"] # 8090 host - avoids clashing with existing services
  networks: [platform-net]

docker compose up -d keycloak
open http://localhost:8090 # Keycloak Admin Console - login admin/admin
```

Create the Realm

```
# Admin Console -> top-left realm dropdown -> Create Realm
# Realm name: ecommerce-platform
#
# A Realm is an isolated space of users, clients, and roles – separate
# from Keycloak's own 'master' realm, used only for Keycloak's own admin.
# Never put application users in the master realm.
```

Realm = isolation boundary

Each realm holds its own independent set of users, clients, and roles. Two realms never share user accounts or configuration.

Create the Gateway Client (Confidential)

```
# Realm 'ecommerce-platform' -> Clients -> Create client
# Client ID: gateway
# Client authentication: ON      <- makes it a CONFIDENTIAL client (has a secret)
# Authentication flow: Standard flow  <- this IS the Authorization Code Flow
# Valid redirect URIs:
#   http://localhost:8080/login/oauth2/code/keycloak
#
# After saving: Credentials tab -> copy the Client Secret
# (needed for the token exchange in Step 5, and for Session 20)
```

Create a Test Customer User

```
# Realm 'ecommerce-platform' -> Users -> Create new user
# Username: customer1
# Email: customer1@example.com
#
# After creating: Credentials tab -> Set password
# 'Temporary' toggle OFF (for smoother live demo)
#
# BONUS at home: create an 'admin' role + separate admin user
# to preview Session 20's role-based access
```

Browser + Postman: Manual Authorization Code Flow

```
# Step A: paste in a browser (this is the redirect the Gateway WOULD do):
http://localhost:8090/realms/ecommerce-platform/protocol/openid-connect/auth
?client_id=gateway&redirect_uri=http://localhost:8080/login/oauth2/code/keycloak
&response_type=code&scope=openid

# Step B: log in as customer1 -> Keycloak redirects with ?code=... in the URL

# Step C: exchange the code for tokens (curl or Postman):
curl -X POST http://localhost:8090/realms/ecommerce-platform/protocol/openid-connect/token \
  -d 'grant_type=authorization_code' -d 'client_id=gateway' \
  -d 'client_secret=<paste-from-Step-14>' -d 'code=<paste-from-Step-B>' \
  -d 'redirect_uri=http://localhost:8080/login/oauth2/code/keycloak'
# Response: access_token, id_token, refresh_token, expires_in
```

Inspecting the Real Keycloak JWT

Demo live: paste the access_token into jwt.io

- header . payload . signature — same structure as before, but look at the payload
- iss — the issuer's URL (Keycloak) — the very thing that lets the Gateway trust WHO signed it
- sub — the user ID, realm_access.roles — the user's assigned roles, exp — expiration

Production note

In a real system, Keycloak or Auth0 issues the tokens. The Gateway only validates — it does not issue tokens. We cover the full OAuth2 Resource Server wiring in Session 20.

Session 3 Hand-Signed JWT vs Keycloak JWT

Claim / Property	Session 3 (hand-signed)	Session 19 (Keycloak)
sub (subject)	hardcoded test value	real user ID
role	manually inserted	realm_access.roles — assigned in Admin Console
iss (issuer)	does not exist	Keycloak's realm URL — enables trust-the-issuer
Signing key	shared HMAC secret	Keycloak's private key (RSA) — Session 20 covers verification

Today: Human Login. NOT Today: Client Credentials.

Today: human login only — the Authorization Code Flow.

A real Identity Provider, a real realm, a real user, and a manually verified end-to-end token exchange.

Session 20 covers the OTHER half.

Service-to-service authentication (Client Credentials Flow) and actually wiring the Gateway as a real OAuth2 Resource Server, replacing JwtAuthFilter for good.

If a question about machine-to-machine auth comes up today, the honest answer is: "great question — that's next session."

Standing Up Keycloak & Running the Flow

Infrastructure + manual flow verification · Embedded across Live Coding blocks · ~55 min

1

Add Keycloak to the Platform

docker-compose.yml + verify Admin Console loads at :8090

2

Create Realm, Client, User

ecommerce-platform realm, Confidential 'gateway' client, 'customer1' user

3

Run and Document the Flow

Build the authorization URL, capture the code, exchange for tokens

4

Compare the Tokens

3-4 sentence note: what fields does this JWT have that Session 3's did not?

What We Added Today

🔍 Capability Added

Real Identity. A Keycloak Identity Provider now exists for the platform, with a genuine realm, a registered Gateway client, and real user accounts.

🔍 Services Updated

Platform infra: Keycloak container added. Realm 'ecommerce-platform' with Gateway (Confidential) client and test customer user. No service code changes yet — Gateway still runs Session 3's JwtAuthFilter until Session 20.

Platform Progress — 8-Segment Tracker



Gateway 🔍 still shows JwtAuthFilter active — Keycloak IdP is standing by, not yet wired to the Gateway. Session 20 completes the swap.



Daily Quiz

8 Questions · 10 Minutes · Google Forms or Kahoot

OAuth2 Roles · Authorization Code Flow · Public vs Confidential · Realms

5 Things That Will Go Wrong Today

"Invalid redirect_uri" error from Keycloak

Must EXACTLY match a Valid Redirect URI on the Client (including trailing slashes). Copy-paste, don't retype.

Token exchange returns 401 invalid_client

Verify client_secret was copied correctly — it regenerates if 'Regenerate' is clicked by accident.

Login page shows but credentials are rejected

Verify the user's password was set with 'Temporary' toggled OFF.

Cannot reach Keycloak Admin Console at all

Keycloak dev-mode image can take 30-60s to fully start — wait and retry.

Token exchange succeeds but no realm_access.roles

Roles must be explicitly assigned to the user — a freshly created user has none by default.



Session 19 Complete

Identity Provider Active — Realm, Confidential Client, Real Users

Session 20: Gateway validation will use Keycloak tokens instead of the custom JwtAuthFilter from Session 3.

Next: Session 20 — Security Part 2
Monday, 3:00–5:30 PM, Online